

User Guide

For AI Enabled Regression Test Optimization

Operating the proof-of-concept application on the nRF52840 bench

Version 1.3 | 25 September 2026

Table of Contents

1. What This Application Demonstrates.....	3
1.1 The two planners	4
1.2 The proof-of-concept bench.....	4
2. System Requirements	5
3. Deliverable Contents	5
4. Installation and Invocation	5
5. What Happens in a Run	6
6. User Interface Reference	7
7. Test Execution Procedure.....	8
7.1 A run with the rule planner.....	8
7.2 A run with the AI planner.....	9
8. Log Artefact Locations	9
9. Processed Audio Artefacts.....	10
10. Fault Diagnosis	10
11. Result Interpretation	11
11.1 Device measurements	11
11.2 Requirement and release-note tests.....	11
11.3 Why tests are skipped.....	12

AI Enabled Regression Test Optimization - User Guide

Version: 1.3

Scope: Operation of the packaged application, AI_Enabled_Regression_Test_Optimization.exe, on the proof-of-concept bench.

Audience: Engineers evaluating AI-enabled regression test optimisation, and test engineers running it against the nRF52840 proof-of-concept board.

Licence: MIT. Copyright (c) 2026 Prajnaana Technologies Pvt. Ltd. Original author: Dhanya Shree S.

The design of the framework, and how to apply it to another product, are in the accompanying Technical Document. Throughout this guide, <install folder> denotes the directory containing AI_Enabled_Regression_Test_Optimization.exe.

1. What This Application Demonstrates

AI Enabled Regression Test Optimization is an open-source reference for two ideas:

- **AI chooses the tests.** Rather than running the whole regression suite on every build, an AI planner reads what changed and the state of the device, and chooses the smallest set of tests, and how hard to run them, that would catch a regression in that change.
- **pytest runs them.** The chosen tests are written as a pytest module, checked, run against real hardware, and reported. Tests are also generated automatically from the product's requirements and from each release note. If a test fails, the application raises the intensity and runs once more.

The framework is not tied to any device. To show that it works on real hardware, this package applies it to a **proof-of-concept target**: a Nordic nRF52840 development board running firmware that stands in for a hearing aid's audio processing, reached over Bluetooth Low Energy. Everything specific to that board is kept in one replaceable layer, shown at the bottom of Figure 1.

Architecture of an AI-enabled regression test suite

Generic core above the line; one adapter per product below it. The nRF52840 board is the proof-of-concept adapter.

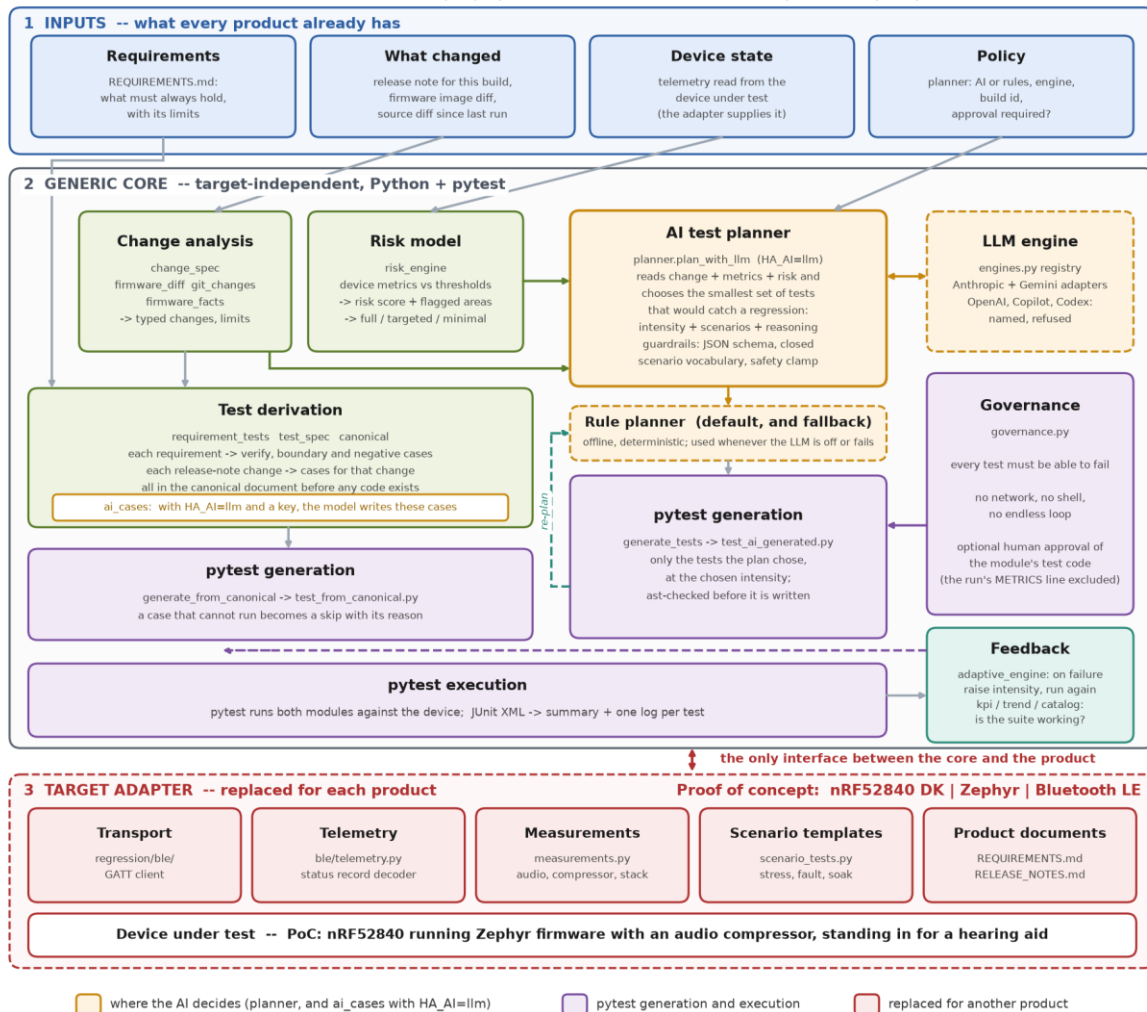


Figure 1 - Architecture of the AI-enabled regression test suite.

1.1 The two planners

The application can choose tests in two ways, selected with the Planner control (section 6):

Planner	Who decides	Needs
With API	The AI planner. A language model reads the change, the device measurements and the risk score, and returns the intensity, the scenarios, and its reasoning. With API, the model also writes the requirement tests (section 11.2).	An API key and an internet connection (section 7.2).
Without API	Fixed rules built into the application. The same inputs, compared with thresholds.	Nothing. This is the default.

Everything else in the run is identical, so the two can be compared on the same build. If the AI planner cannot be used for any reason, the run falls back to the rules and says so.

1.2 The proof-of-concept bench

On the bench, audio is streamed from the computer to the board over Bluetooth, processed there by a compressor, and sent back. The audio tests compare what came back against what was sent. The board also reports

measurements of its own state, such as memory use and link drops, which the application uses to judge the risk of the build.

2. System Requirements

Item	Requirement
Operating system	Windows 11, 64-bit.
Python	Not required. The Python runtime and every dependency, including the AI planner's client, are inside the package.
Host radio	A Bluetooth Low Energy adapter, enabled in Windows Settings. Bluetooth Classic alone is not sufficient.
Target board	Proof of concept: an nRF52840 development kit running the supplied firmware, powered on and advertising. Boards whose name contains "Zephyr" are listed first; any board can be chosen.
Disk space	Approximately 100 MB for the package, plus about 1 MB for each run's logs.
Write access	The application writes into <install folder>. Install somewhere the account can write to, not Program Files. If the folder is not writable, results go to %LOCALAPPDATA%\AI Enabled Regression Test Optimization instead.
Network and API key	Needed only for the With API planner (section 7.2).

3. Deliverable Contents

These items sit in <install folder> and must stay together; the application will not start if _internal is moved or renamed.

Item	Size	What it is
AI_Enabled_Regression_Test_Optimization.exe	12 MB	The application. Double-click to start it. No command window opens.
_internal\	73 MB	Everything the application needs to run: the Python runtime, the window toolkit and the native libraries. The pure-Python parts, including the model client and the test runner, are inside the .exe. Not intended to be opened or edited.
REQUIREMENTS.md	10 KB	The requirements of the proof-of-concept device. Each requirement becomes tests on every run. See section 11.
NRF_Firmware\RELEASE_NOTES.md	10 KB	What changed in each firmware build of the proof of concept. The section for the build on the board becomes tests on every run. See section 11.
THIRD_PARTY_NOTICES.txt	264 KB	The licences of the third-party components inside the package.
User_Guide.pdf	1 MB	This guide.
regression\	created on first run	Where the application writes its results. See section 8.

4. Installation and Invocation

Power the board and check it is advertising, then double-click AI_Enabled_Regression_Test_Optimization.exe. The window shown in section 6 appears after a few seconds. The first start is slower, because Windows checks the package. There is no installation step and no administrator rights are needed. To move the application, move the whole folder.

Note. Where Smart App Control or a similar policy is switched on, an unsigned application can be blocked the first time it runs, or shortly after it has been copied to a new folder. If it does not start, or reports that a component failed to load, the package has to be allowed by the local security policy.

5. What Happens in a Run

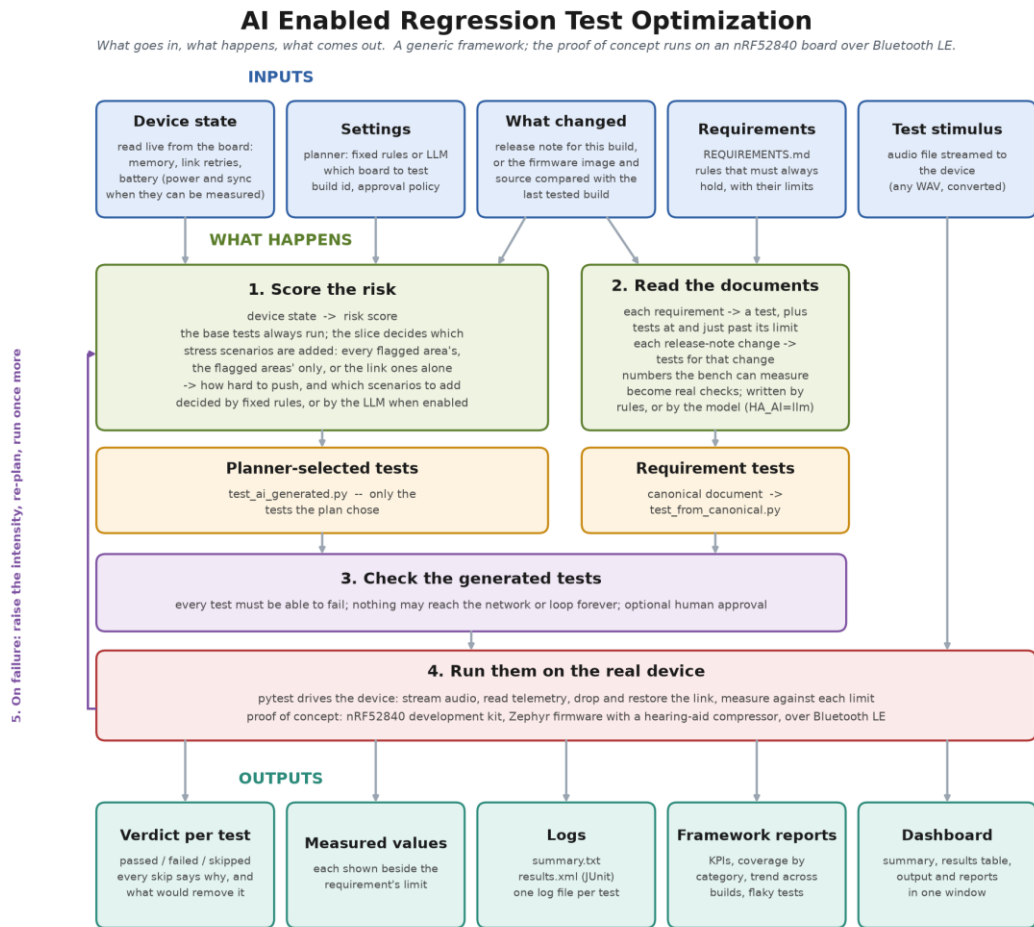


Figure 2 - One run: what goes in, what happens, what comes out.

1. **Work out what changed.** The section of the release note for the build on the board, if a release note is selected. Otherwise the application compares the firmware with the last build it tested. A change it cannot work out is reported as unknown; the plan then rests on the board's measurements alone.
2. **Read the board.** Memory use, link drops and battery level. Supply current and binaural sync are read only where an instrument or a second board exists.
3. **Score the risk** and choose how much to run: everything, only the flagged areas, or just the connection.
4. **Plan.** The AI planner, or the rules, chooses the intensity (low, medium or high) and any stress scenarios to add. The Output box shows "Selected N of M tests" and the tests the plan left out.
5. **Write the tests** as two pytest modules: one from the plan, one from the requirements and release note.
6. **Check the tests.** Every test must be able to fail, and none may reach the network or loop forever.
7. **Run them** against the board and show each result as it finishes.
8. **Escalate on failure.** Raise the intensity, plan again, and run once more.
9. **Record the run** so the Framework tabs can report on the framework itself.

6. User Interface Reference

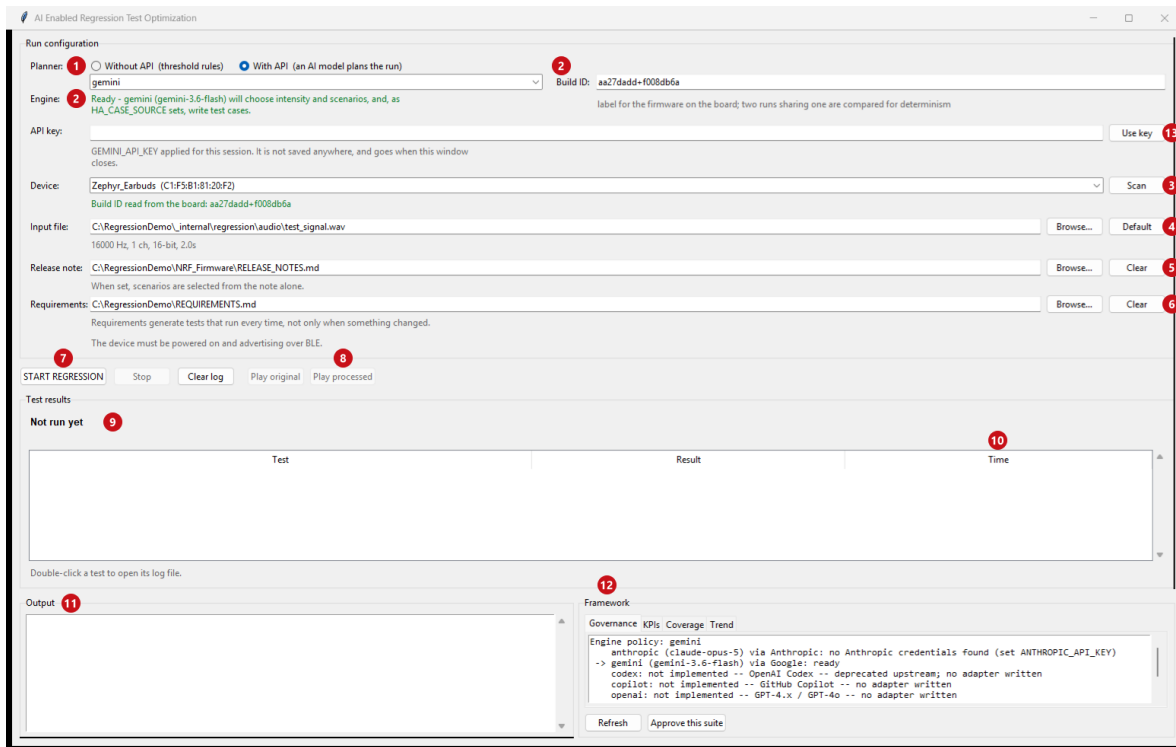


Figure 3 - The window with With API selected, before a run. The Build ID shown is the firmware current when the screenshot was taken; the field reads whatever the board reports on the day.

#	Control	Purpose
1	Planner	Chooses who selects the tests. With API asks the AI planner (section 1.1). Without API uses the fixed rules and needs no key and no internet. The line under the Engine box says whether the chosen option is available (grey for rules, green "Ready - ...", amber "Unavailable: ... fall back to rules", red if the engine cannot be loaded); if it is not, the run uses the rules.
2	Engine and Build ID	Engine picks which AI model plans the run when Planner is With API; it is greyed out under Without API. Two engines are implemented: anthropic (the default) and gemini. Engines that are not implemented appear as e.g. "openai (no adapter)" and are refused if chosen. The API key box appears under Engine on the With API path only; it is described at row 13. Build ID labels the firmware under test. Choosing a board reads its id into this field ("Build ID read from the board: ..."); if the board reports none, the id from the last run stays, and you can type one. The release-note section with this id is the one used. Runs with the same Build ID are compared.
3	Device and Scan	The board under test. It starts at "-- select a device --" and a board must be chosen before a run can start; the application never picks one on the operator's behalf. Press Scan to list nearby Bluetooth devices, then choose the board. Boards whose name contains "Zephyr" are sorted to the top.
4	Input file	The audio sent to the board. It starts as the built-in test signal, or as sample_sound.wav if you have placed a WAV of that name in <install folder>. Browse picks a different WAV; Default goes back to the starting file. The grey line underneath reports the file's format and any conversion applied; any WAV is mixed to mono and resampled to 16000 Hz automatically. An unreadable file is reported in red ("File not found." or "Not a usable WAV: ...").
5	Release note	The release note describing what changed in the firmware. It starts as NRF_Firmware\RELEASE_NOTES.md. Browse picks another; Clear leaves it out, and the change is then worked out from the firmware. In the packaged application there is no firmware image to compare, so the change is then reported as unknown. See section 11.
6	Requirements	The requirement document the run tests against. It starts as REQUIREMENTS.md. Browse picks another; Clear leaves requirements out. See section 11.

7	START REGRESSION, Stop, Clear log	START begins a run. Stop ends it early. Clear log empties the Output box. START is greyed out while a run is going on.
8	Play original, Play processed	Plays the before and after recordings described in section 9. Both stay greyed out until those two files exist beside the application.
9	Run summary	Says "Not run yet" before the first run, and "Running..." with a running count of passed, failed and skipped tests during one. Summary line (top of Test results): "PASS risk score N slice FULL/TARGETED/SANITY intensity ... planner rules/llm". The number of tests selected is the "Selected N of M tests" line in Output. The line below shows power, memory, sync and retry. A run that did not finish shows "Did not finish - check the log (is the device advertising?)".
10	Test results	One line per test with its result and duration, filled in as each test finishes. Green passed, red failed, amber skipped. Double-click a line to open that test's log.
11	Output	Everything the run printed, in order, including the planner's reasoning for its choice.
12	Framework	Four tabs about the framework itself, refreshed after every run: Governance (whether the generated tests pass the acceptance rules, and who approved them), KPIs, Coverage (tests per category) and Trend (this run against earlier ones). Approve this suite records your approval of the tests just generated, and approves both generated modules (test_ai_generated.py and test_from_canonical.py); the panel lists both. The tab also lists the engines and the power source.
13	API key	Only appears when Planner is With API; the row is hidden under Without API. It is the only place to type a key into the application; a key already set in the Windows environment is used as well. The box is masked and starts empty. Type or paste the key and press Use key; the box is cleared once the key is applied, and the key applies to the runs that follow in this session. If the key looks like another engine's - a Gemini key pasted while Engine is anthropic, for example - the line underneath says so and applies it anyway. The key is never saved anywhere: it is held in the running application, passed to the runs it starts, and goes when the window closes - so it cannot be left behind on a shared machine and cannot travel when the folder is copied. The grey line beneath then reads "ANTHROPIC_API_KEY applied for this session. It is not saved anywhere, and goes when this window closes.", without ever showing the key itself. See section 7.2.

7. Test Execution Procedure

7.1 A run with the rule planner

Step	Action	Expected behaviour
1	Switch on the board and check it is advertising.	The board is discoverable.
2	Start AI Enabled Regression Test Optimization.exe.	The window opens and the summary says "Not run yet".
3	Set Planner to Without API.	The line under Engine says "Offline, free, deterministic. No API key needed."
4	Press Scan and pick the board.	Scan reports "Found N devices (M named)". Choosing the board shows "Build ID read from the board: ...". Pressing START before choosing a board shows "Select the device to test before starting".
5	Leave Input file on the default, or Browse for another WAV.	The grey line describes the file and any conversion.
6	Check the Release note and Requirements fields.	They show NRF_Firmware\RELEASE_NOTES.md and REQUIREMENTS.md from <install folder>.
7	Press START REGRESSION.	The Output box fills and result lines appear one by one.
8	Wait for the run to end.	The summary shows PASS or FAIL. A run usually takes about 100 seconds.
9	Double-click any result line.	That test's log file opens.

If the summary says the run did not finish, the board was not reachable. Check it is switched on and advertising, and that nothing else is connected to it. The firmware accepts one connection at a time.

7.2 A run with the AI planner

The AI planner needs an API key for the model engine. The key is entered in the application window; there is no configuration file to put one in, and it is never saved to disk. Get an Anthropic key from the Anthropic Console, or a Gemini key from Google AI Studio (aistudio.google.com), which has a free tier. A key already set in the Windows environment as ANTHROPIC_API_KEY or GEMINI_API_KEY is used too, and the line under API key then says it is set. GOOGLE_API_KEY also works for gemini, but that line does not report it; the line under Engine does.

1. Set Planner to With API, and choose the Engine: anthropic (the default) or gemini.
2. Paste the key into the API key row that appears under Engine, and press Use key. It applies to this session only: the key is held while the window is open, passed to the runs it starts, and goes when you close it. Nothing is saved, so there is nothing to clear afterwards.
3. Read the line under the API key box: it confirms the key was applied, or warns that it looks like another engine's. The line under Engine then reports whether the engine is ready. If it does warn, either change Engine to match the key or paste the right one.

Then follow section 7.1, skipping step 3, because the Planner is already set. The line under Engine should read "Ready - anthropic (claude-opus-5) will choose intensity and scenarios, and, as HA_CASE_SOURCE sets, write test cases.", naming whichever engine and model you chose. HA_CASE_SOURCE is an environment setting; left unset, the model writes the requirement and release-note cases whenever a key reaches it. After the run, the summary shows "planner llm" (or "planner rules" if it fell back), and the Output box shows the model's reasoning for the intensity and scenarios it chose.

The gemini engine is the one that has been exercised against a live key: it planned complete runs, and its failure paths were checked as well, so a model that is unreachable, retired or out of its daily quota leaves the run to finish on the rule planner, with the reason printed in the Output box. Two limits stand: the anthropic engine has not been run against a live key, and generating cases from the whole requirement document has not completed against a live model, because the Gemini free tier refuses a request that large.

Comparing the planners. Run the same build once with each planner, with the same Build ID. Compare the "Selected N of M tests" line in Output, and summary.txt in each run's folder (section 8). The Test results table is cleared at each START.

What is sent. The planner sends the board's measurements, the change description, the risk score and a summary of the last five runs. With API also lets the model write the requirement tests, and for that the requirement document and the release-note section for this build are sent. No audio and no firmware leave the computer. Without API sends nothing anywhere. SECURITY.md in the project repository describes this in full.

8. Log Artefact Locations

Everything is written inside <install folder>. Nothing goes to the registry or to a temporary folder. There is one exception: if <install folder> is not writable, the same tree is written under %LOCALAPPDATA%\AI_Enabled_Regression_Test_Optimization instead. No API key is written in that folder or in any other. The key you type is held in the running application and goes when the window closes, so it never appears in a log, an artefact or a file of any kind.

Path	Contents
<install folder>\regression\logs\build_<date>_<time>\	One folder per run, named after the time it started. Folders are never deleted by the application.
summary.txt	One line per test: name, result, and duration.
results.xml	The same results in JUnit XML, the format test tracking tools

	and build servers read.
tests\<test name>.log	One file per test with its full output and, if it failed, the reason. The file to attach to a bug report.
regression\generated_tests\test_ai_generated.py	The pytest module written from the plan for the most recent run: exactly which tests were chosen and the values they used.
regression\generated_tests\test_from_canonical.py	The pytest module written from the requirements and release note. Every skipped test in it states why.
regression\artifacts\	KPI history, firmware fingerprints and approvals.
regression\generated_tests\confest.py	The session fixture that gives each generated test a connected board.

A run that escalates writes a second build_ folder.

9. Processed Audio Artefacts

Two files hold the audio from either side of the proof-of-concept board, both in <install folder> (or the fallback folder of section 8):

File	Contents
dsp_before.wav	The audio sent to the board.
dsp_after.wav	The audio the board sent back after processing it.

Play original and Play processed play these files, and stay greyed out until they exist. Playing one after the other is the way to hear the compressor working: loud parts should be quieter in the processed file, while quiet parts stay close to unchanged.

Recording in this release. The packaged application does not record: it has no tick box for it, because the script that makes the recordings, dsp_listen.py, is not shipped with it. To produce the two files, run that script from a source installation and copy dsp_before.wav and dsp_after.wav into <install folder>. Play original and Play processed then work.

10. Fault Diagnosis

Observation	Reason	Procedure
The application does not start, or says a component failed to load.	The package is incomplete, or Windows is blocking it.	Check _internal is beside the application. Allow the application in the security policy.
The summary says the run did not finish.	The board was not reachable.	Check it is powered and advertising. Press Scan to see whether it appears.
Scan finds nothing, or fails.	The computer's Bluetooth is off or missing.	Switch Bluetooth on in Windows Settings.
Scan lists the board, but the run cannot connect.	Something else is connected to it, or Windows remembers an old version of the board's services.	Close the other program. If it persists, the board needs reflashing with a changed Bluetooth address; see section 14.3 (Identity address) of the Technical Document.
With API is selected but the run used the rules.	No API key was found, there is no internet connection, or the model could not be reached.	Read the line under Engine and the [planner] line in the Output box. Enter the key in the API key row and press Use key (section 7.2).
Some tests show SKIPPED.	The test cannot be judged on this bench. The reason is in its log and in the Output box.	Read the reason; section 11 lists each one and what removes it. If all the audio tests skip together, the firmware does not publish the processed-audio characteristic: reflash the board with the supplied firmware.
The red line under Input file says "File not found." or "Not a usable WAV: ..."; START then says "Pick	The file is not a WAV the application can read.	Choose another file, or press Default.

a readable WAV file before starting".		
---------------------------------------	--	--

11. Result Interpretation

11.1 Device measurements

test_memory and test_retry compare numbers the board reports against limits. test_power and test_sync are skipped on this bench: the board cannot measure its own supply current, and sync is the timing between a left and a right hearing aid while the bench has one board. A skipped test has not passed; the summary counts it separately and its log gives the reason.

The audio tests and the requirement tests are real measurements of the board. A requirement test prints the measured value next to its limit, for example: requirement: latency_ms < 2000 ms; measured 63 ms.

11.2 Requirement and release-note tests

Besides the tests chosen by the planner, every run writes tests from two plain-text documents in <install folder>: REQUIREMENTS.md, what the device must always meet, and NRF_Firmware\RELEASE_NOTES.md, what changed in each firmware build. Both describe the proof-of-concept device; another product supplies its own. Edit either in any text editor and the change takes effect at the next run.

Writing a requirement. One per line: an identifier, a colon and one sentence, for example "REQ-AUD-004: Host-to-device-and-back latency shall be under 2000 ms." The identifier must start with REQ-, SRS-, FR- or NFR-. The middle part of the identifier names the part of the device: BLE, AUD (audio), ANC (noise cancellation), BAT (battery), MEM (memory), PWR (power), SYN (sync), TLM (telemetry), SEC (security) or SEN (sensor). Other lines, such as headings and notes, are ignored.

What a requirement produces. Each requirement becomes a Verify test. If it states a limit, such as "under 2000 ms", "at least 25 percent" or "between 0 and 100 percent", it also becomes tests at the limit and just past it. Removing a requirement removes its tests; adding one adds them. With API, the model writes these tests instead; if it returns none, the rule-derived tests stand.

Requirements the bench measures. A requirement using one of these phrases and stating a number is checked against that number:

The requirement mentions	What is measured
signal-to-noise	Signal-to-noise ratio of the processed audio, in dB.
latency	Time from sending audio to the first processed audio coming back, in ms.
gain for quiet input	How much more quiet sound is amplified than loud sound, in percent.
applied gain shall remain	The lowest and highest gain the compressor applies.
opposite sign	Percent of loud samples that come back inverted.
attack, release	The compressor's attack and release times, in ms. State each as "within N ms".
battery level shall be between	Battery level, in percent.
stack high-water	Memory stack in use, in KiB.
build identifier	Length of the firmware build id, in characters.

A requirement about something the bench cannot measure still produces tests, but they are skipped with the reason.

Writing the release note. Add a section for each build, headed ## followed by the build id the board reports, newest first. Only the section whose heading contains the board's build id is read. If no heading contains the full id, a heading containing its +hash part (for example +f008db6a) is used. If no heading matches either, the whole note is read. Write one bullet per change and name the part of the device it affects, such as

battery, Bluetooth, audio or memory; a bullet under a **### Battery** or **### Bluetooth** heading counts as naming it.

11.3 Why tests are skipped

The skip says	What it means	What removes it
...cannot make the device produce exactly ...	A test at or just past a limit. The value can be measured, but the board cannot be forced to that exact value.	Firmware built to misbehave on command (fault injection).
power not measured ..., or needs an external analyzer; the board cannot measure its own supply	The board cannot measure its own supply current.	A PPK II power analyser (HA_POWER_MA or HA_POWER_CSV).
sync not measured: binaural sync ..., or binaural sync needs two devices; this bench has one	Sync is the timing between a left and a right hearing aid. The bench has one board.	A second board.
Needs a second Bluetooth central ...	The requirement allows one connection at a time, and testing that limit needs a second computer or adapter to attempt the second one.	A second Bluetooth central.
The firmware has no heap ...	The firmware never allocates memory while running.	Nothing needed.
device does not implement audio loopback ... it publishes no processed-audio notify characteristic, so it accepts the audio and notifies nothing back	The firmware does not publish the processed-audio characteristic, so it takes the audio and sends nothing back and there is nothing to measure. A board that publishes it but then stops answering is a failure, not a skip.	Reflash the board with the supplied firmware.
Build-system change ...	A change to how the firmware is built, not to what it does.	Nothing needed.
The release-note line ... does not name a part of the device	The bullet does not say which part it affects.	Name the part: battery, Bluetooth, audio, memory, power, telemetry or sensor.
no temperature interface on this target	The development kit has no temperature sensor.	Hardware with a temperature sensor.
no interface for this component on this bench	The bench has no way to drive that part of the device - noise cancellation, for example.	Firmware and a bench interface for that part.
the device accepted 0 bytes ...	The board accepts an empty audio write that the requirement implies it should refuse.	A decision on whether the requirement or the firmware is right.
Not a real test: 'unknown' is not a part of the device	A placeholder standing for release-note lines that name no testable part	Nothing needed.